

Random Encounter Generator — Implementation Plan

Project: D&D 3.5e Prototype (dnd35prototype)

Phase: Phase 5 — Random Encounter Generator

Companion: [encounter_generator_design.md](#) (encounter_generator_design.md)

Date: 2026-05-26

Status: PLAN ONLY — No implementation code

Table of Contents

- 1. [Implementation Phases](#)
- 2. [Phase 5.0: Preparation](#)
- 3. [Phase 5.1: DiceExpression](#)
- 4. [Phase 5.2: EncounterDescriptionParser](#)
- 5. [Phase 5.3: EncounterCSVParser & Table Builder](#)
- 6. [Phase 5.4: Core Integration](#)
- 7. [Phase 5.5: UI & Polish](#)
- 8. [Phase 5.6: Testing & Validation](#)
- 9. [File Manifest](#)
- 10. [Risk Register](#)
- 11. [Effort Estimates](#)
- 12. [Migration & Rollback Plan](#)
- 13. [Definition of Done](#)
- 14. [Dependencies & Prerequisites](#)

1. Implementation Phases

Phase 5.0: Preparation	 CSV completion, branch setup
Phase 5.1: DiceExpression	 Standalone dice parser/roller
Phase 5.2: DescriptionParser	 Encounter string parser
Phase 5.3: CSVParser + Builder	 CSV loading  table construction
Phase 5.4: Core Integration	 Wire into existing manager + entry classes
Phase 5.5: UI & Polish	 Level slider update, encounter display
Phase 5.6: Testing & Validation	 Comprehensive validation pass

Each phase produces a **compilable, non-breaking** commit. Earlier phases can merge independently.

2. Phase 5.0: Preparation

Goal: Ensure all prerequisites are in place before coding begins.

Tasks

#	Task	Details
5.0.1	Complete Table 9 CSV data	Current CSV truncated at row 43-44 for level 9. Transcribe remaining entries from DMG images (randomdungeon2.png)
5.0.2	Validate CSV data integrity	Check all 9 tables have cascade entries at both ends; verify d% ranges cover 1-100 per table without gaps/overlaps
5.0.3	Create feature branch	<code>git checkout -b feature/phase5-encounter-generator</code> from <code>master</code>
5.0.4	Copy CSV to StreamingAssets	<code>cp dungeon_encounters.csv Assets/StreamingAssets/dungeon_encounters.csv</code>
5.0.5	Creature gap audit	Cross-reference all CSV creature names against <code>_creatureNameMap</code> entries; list any missing mappings

Files Modified

- `dungeon_encounters.csv` — Complete table 9 data
- `Assets/StreamingAssets/dungeon_encounters.csv` — New file (copy)

Acceptance Criteria

- [] CSV has complete data for all 9 tables
- [] Every table's d% ranges cover 1-100 without gaps
- [] CSV copied to StreamingAssets

Effort: ~1 hour

3. Phase 5.1: DiceExpression

Goal: Create a standalone, well-tested dice expression parser and roller.

Tasks

#	Task	Details
5.1.1	Create <code>DiceExpression.cs</code>	New file in <code>Assets/Scripts/Encounters/</code>
5.1.2	Implement constructor	Private constructor, readonly fields: <code>NumDice</code> , <code>DiceSides</code> , <code>Modifier</code> , <code>Original</code>
5.1.3	Implement <code>Parse(string)</code>	Static factory using regex <code>^(\d+)(?:d(\d+)([+-]\d+)?)?\$</code>
5.1.4	Implement <code>Roll()</code>	Uses <code>UnityEngine.Random.Range(1, sides+1)</code> per die, adds modifier, floors at 1
5.1.5	Implement properties	<code>IsFixed</code> , <code>Minimum</code> , <code>Maximum</code> , <code>ToString()</code>
5.1.6	Add XML doc comments	Full documentation per project conventions
5.1.7	Manual test	Call <code>DiceExpression.Parse()</code> for each pattern found in CSV; verify no nulls

File Created

`Assets/Scripts/Encounters/DiceExpression.cs`

Code Skeleton

```
using System;
using System.Text.RegularExpressions;
using UnityEngine;

/// <summary>
/// Immutable dice expression supporting "NdS+M" notation and fixed values.
/// Phase 5: Random Encounter Generator.
/// </summary>
[Serializable]
public class DiceExpression
{
    public readonly int NumDice;
    public readonly int DiceSides;
    public readonly int Modifier;
    public readonly string Original;

    // Constructor, Parse, Roll, IsFixed, Minimum, Maximum, ToString
}
```

Test Cases

Input	Expected NumDice	Sides	Modifier	Min	Max
"1"	0	0	1	1	1
"3"	0	0	3	3	3
"1d3"	1	3	0	1	3
"1d4"	1	4	0	1	4
"2d4"	2	4	0	2	8
"1d3+1"	1	3	1	2	4
"2d4+1"	2	4	1	3	9
"1d4+4"	1	4	4	5	8
"2d4+3"	2	4	3	5	11
" "	null	-	-	-	-
"abc"	null	-	-	-	-

Acceptance Criteria

- [] All test cases pass
- [] Roll() returns values within [Minimum, Maximum] over 10,000 trials
- [] No external dependencies beyond System and UnityEngine

- [] Compiles without warnings

Effort: ~30 minutes

4. Phase 5.2: EncounterDescriptionParser

Goal: Parse all encounter description patterns from the CSV into structured data.

Tasks

#	Task	Details
5.2.1	Create <code>EncounterDescriptionParser.cs</code>	New file with static methods
5.2.2	Create <code>ParsedCreatureGroup</code> class	Inline in same file or separate — stores one creature group
5.2.3	Create <code>ParsedEncounterDescription</code> class	Container for parsed result
5.2.4	Implement cascade detection	Regex: <code>^Roll on (\d+)(?:st nd rd th)-level table\$</code>
5.2.5	Implement compound splitting	Split on <code>\s+and\s+(?=\d)</code> — only split when right side starts with digit
5.2.6	Implement NPC pattern	Regex for <code>Nth-level {race} {class} NPC</code> patterns
5.2.7	Implement standard creature pattern	Count + name + optional annotation extraction
5.2.8	Implement special comma patterns	Handle "1 ghost, 5th-level fighter" style entries
5.2.9	Implement annotation stripping	Extract <code>(vermin)</code> , <code>(demon)</code> , etc. from creature names
5.2.10	Implement plural normalization	Strip trailing 's'/'es' for name resolution
5.2.11	Validate against all CSV rows	Run parser over every row, log any failures

File Created

```
Assets/Scripts/Encounters/EncounterDescriptionParser.cs
```

Pattern Priority Order

The parser should try patterns in this order (first match wins):

- 1. **Cascade**: "Roll on Nth-level table"
- 2. **NPC**: "[count] Nth-level {race} {class} NPC[s]"
- 3. **Special**: "{count} {creature}, Nth-level {class}" (comma-separated)
- 4. **Standard**: "{dice_expr} {creature_name} [(annotation)]"
- 5. **Fallback**: Entire string as description, count=1, warn

Critical Edge Cases

CSV Entry	Parsing Notes
"1 hobgoblin warrior and 1d4 goblin warriors"	Compound: split on " and ", two groups
"5th-level lizardfolk druid NPC (with crocodile)"	NPC + companion annotation — parse NPC, note companion
"1 formian taskmaster and 1 dominated 5th-level human barbarian NPC"	Compound: standard + NPC with "dominated" modifier
"1d3 devils, hellcat"	Comma variant spec — parse as 1d3 hellcat devils
"1 ghost, 5th-level fighter"	Special comma pattern: creature + class levels
"1 half-dragon 4th-level fighter"	Template + class — parse template "half-dragon", class "fighter" level 4
"1 five-headed hydra"	Descriptive adjective — name map should handle "five-headed hydra" → "hydra_5"
"1d3+1 violet fungi and 1d3+2 shriekers (fungus)"	Compound with annotation on second group only

Acceptance Criteria

- [] Parses all ~267 CSV encounter descriptions without exceptions
- [] Cascade entries detected for all "Roll on..." patterns
- [] Compound entries correctly split into multiple groups
- [] NPC entries correctly extract race, class, and level
- [] Annotations stripped and preserved separately
- [] Parse warnings < 5% of total rows

Effort: ~2 hours

5. Phase 5.3: EncounterCSVParser & Table Builder

Goal: Load CSV file and construct `DungeonEncounterTable` objects from parsed data.

Tasks

#	Task	Details
5.3.1	Create <code>EncounterCSVParser.cs</code>	Static class for CSV reading
5.3.2	Implement CSV line parser	Handle quoted fields (commas in descriptions)
5.3.3	Implement <code>ParseCSV(string path)</code>	Returns <code>List<RawEncounterRow></code>
5.3.4	Add <code>BuildFromCSV()</code> to <code>DungeonEncounterTableData.cs</code>	Converts parsed rows into <code>Dictionary<int, DungeonEncounterTable></code>
5.3.5	Implement creature name resolution	Use existing <code>_creatureNameMap</code> + fallback logic
5.3.6	Implement entry builder	<code>ParsedEncounterDescription</code> → <code>DungeonEncounterTableEntry</code>
5.3.7	Add new name mappings	Add mappings for any CSV names not yet in <code>_creatureNameMap</code>
5.3.8	Validate built tables	Check d% coverage, entry counts, creature ID resolution

Files Created/Modified

NEW: `Assets/Scripts/Encounters/EncounterCSVParser.cs`
 MOD: `Assets/Scripts/Encounters/DungeonEncounterTableData.cs` (add `BuildFromCSV` method)
 MOD: `Assets/Scripts/Encounters/DungeonEncounterTableManager.cs` (add name mappings)

Tasks

#	Task	Details
5.4.1	Add <code>CountExpression</code> to <code>EncounterCreatureEntry</code>	New <code>DiceExpression</code> field
5.4.2	Add <code>ResolveCount()</code> to <code>EncounterCreatureEntry</code>	Rolls dice, sets <code>Count</code>
5.4.3	Update <code>ToEncounterDefinition()</code>	Call <code>ResolveCount()</code> on each cloned creature entry
5.4.4	Update <code>MaxLevel</code> constant	<code>DungeonEncounterTableManager.MaxLevel</code> from 8 to 9
5.4.5	Update <code>LoadTables()</code>	Try CSV first, fallback to hardcoded
5.4.6	Update cascade boundary	Level 9 cascades harder → wraps to 9 (not 8)
5.4.7	Ensure backward compatibility	Existing hardcoded entries (Phase 3) still work unchanged
5.4.8	Integration test	Generate 100 random encounters per level, verify no errors

Files Modified

```
MOD: Assets/Scripts/Encounters/EncounterDefinition.cs
- Add CountExpression field + ResolveCount() to EncounterCreatureEntry

MOD: Assets/Scripts/Encounters/DungeonEncounterTableEntry.cs
- Update ToEncounterDefinition() to resolve dice

MOD: Assets/Scripts/Encounters/DungeonEncounterTableManager.cs
- MaxLevel = 9
- LoadTables() CSV-first logic
- Cascade boundary update for level 9
```

Backward Compatibility Strategy

```

IF csv file exists:
  Load tables from CSV (levels 1-9)
  Tables support dice expressions → resolved at generation time
ELSE:
  Load hardcoded tables (levels 1-8)
  Tables use fixed counts → no dice resolution needed
  (CountExpression is null → ResolveCount() is a no-op)
END IF

In either case:
  GenerateRandomEncounter() works identically
  ToEncounterDefinition() always returns concrete integer counts
  DungeonEncounterSpawner sees no difference

```

Acceptance Criteria

- [] `GenerateRandomEncounter(1-9)` produces valid `EncounterDefinition` from CSV
- [] `GenerateRandomEncounter(1-8)` still works when CSV is absent (hardcoded fallback)
- [] Dice-based counts produce variable results across multiple generations
- [] Compound entries produce `EncounterDefinition` with multiple `Entries`
- [] NPC entries produce entries with correct `TemplateClass` and `TemplateLevel`
- [] No compile errors or warnings

Effort: ~1.5 hours

7. Phase 5.5: UI & Polish

Goal: Update the UI to support 9 levels and improve encounter display.

Tasks

#	Task	Details
5.5.1	Update level slider max	DungeonEncounterGeneratorUI.cs : slider max from 8 to 9
5.5.2	Update level range labels	Any hardcoded "1-8" text → "1-9"
5.5.3	Enhance encounter display	Show dice expression in description (e.g., "1d3+1 goblins → rolled 3")
5.5.4	Add CSV load status indicator	Show whether tables loaded from CSV or hardcoded
5.5.5	Update GameManager.DungeonEncounters.cs	Any level range references

Files Modified

MOD: Assets/Scripts/UI/DungeonEncounterGeneratorUI.cs
 MOD: Assets/Scripts/Encounters/GameManager.DungeonEncounters.cs

Acceptance Criteria

- [] Level 9 selectable in UI
- [] Encounter descriptions display correctly for dice-based counts
- [] CSV/hardcoded status visible to developer (Debug.Log at minimum)

Effort: ~30 minutes

8. Phase 5.6: Testing & Validation

Goal: Comprehensive validation that the system works correctly end-to-end.

Tasks

#	Task	Details
5.6.1	Parser coverage test	Run parser over all CSV rows, verify 0 exceptions
5.6.2	Name resolution audit	List all creature names that fail to resolve; add missing mappings
5.6.3	Dice range validation	For each dice expression in CSV, verify it parses and produces valid ranges
5.6.4	Table coverage test	For each table 1-9, verify d% range 1-100 fully covered
5.6.5	Generation stress test	Generate 1000 encounters per level, verify no nulls or exceptions
5.6.6	Spawn pipeline test	Verify generated Encounter-Definition objects pass through PrepareEncounter()
5.6.7	Cascade test	Verify cascades work at levels 1 (only up) and 9 (only down/wrap)
5.6.8	Regression test	Verify Phase 3 hardcoded tables still produce same results when CSV absent
5.6.9	Visual QA	In Unity Play mode: generate encounters at all 9 levels, verify spawns

Test Script (Editor Mode)

```
// Assets/Scripts/Editor/EncounterGeneratorTests.cs
[MenuItem("Debug/Test Encounter Generator")]
public static void RunTests()
{
    // Test 1: DiceExpression parsing
    // Test 2: Description parsing coverage
    // Test 3: CSV loading
    // Test 4: Table validation
    // Test 5: Generation stress test
    // Test 6: Cascade boundary test
}
```

Acceptance Criteria

- [] 0 exceptions across all tests
- [] Creature name resolution $\geq 90\%$
- [] All 9 tables pass d% validation
- [] 1000 encounters per level generated without error
- [] Cascade works correctly at boundary levels

Effort: ~1.5 hours

9. File Manifest

New Files (3)

File	Phase	Description
Assets/Scripts/Encounters/DiceExpression.cs	5.1	Dice notation parser and roller
Assets/Scripts/Encounters/EncounterDescriptionParser.cs	5.2	Encounter description string parser
Assets/Scripts/Encounters/EncounterCSVParser.cs	5.3	CSV file reader and raw row parser

Modified Files (5)

File	Phase	Changes
Assets/Scripts/Encounters/EncounterDefinition.cs	5.4	Add CountExpression + ResolveCount() to Encounter-CreatureEntry
Assets/Scripts/Encounters/DungeonEncounterTableEntry.cs	5.4	Update ToEncounterDefinition() for dice resolution
Assets/Scripts/Encounters/DungeonEncounterTableData.cs	5.3	Add BuildFromCSV() method
Assets/Scripts/Encounters/DungeonEncounterTableManager.cs	5.4	MaxLevel=9 , CSV loading, cascade boundary
Assets/Scripts/UI/Dungeon-EncounterGeneratorUI.cs	5.5	Level slider max 8→9

Data Files (1)

File	Phase	Description
Assets/StreamingAssets/dun-geon_encounters.csv	5.0	Complete encounter CSV (copy from project root)

Optional Test File (1)

File	Phase	Description
Assets/Scripts/Editor/EncounterGeneratorTests.cs	5.6	Editor-mode test runner

Total: 3 new code files + 5 modified + 1 data file + 1 optional test = 10 files

10. Risk Register

#	Risk	Impact	Likelihood	Mitigation
R1	CSV parsing edge cases break on unexpected formats	Medium	Medium	Extensive pattern catalog (see design doc §4); fallback to description-only entries
R2	Creature names not in NPCDatabase	Low	High	Graceful fallback (log + skip); separate creature-addition phases handle gaps
R3	Table 9 CSV data incomplete	Medium	Known	Transcribe from DMG images in Phase 5.0
R4	Dice rolling produces 0 creatures	Low	Low	Floor at 1 in <code>DiceExpression.Roll()</code>
R5	Compound entries overwhelm spawn points	Low	Low	Spawner already handles overflow via positioning fallback
R6	Regex parsing performance	Negligible	Low	Parsed once at load time; regex compiled once
R7	Breaking existing hardcoded tables	High	Low	CSV loading is opt-in; hardcoded tables untouched; fallback logic
R8	" and " split breaks creature names containing "and"	Medium	Low	Split regex requires digit after "and"; no creature names start with a digit

11. Effort Estimates

Phase	Description	Estimate	Cumulative
5.0	Preparation	1 hour	1 hour
5.1	DiceExpression	30 min	1.5 hours
5.2	EncounterDescriptionParser	2 hours	3.5 hours
5.3	EncounterCSVParser + Builder	2 hours	5.5 hours
5.4	Core Integration	1.5 hours	7 hours
5.5	UI & Polish	30 min	7.5 hours
5.6	Testing & Validation	1.5 hours	9 hours
	Total	~9 hours	

Note: These estimates assume the developer is familiar with the existing codebase. First-time contributors should add ~30% buffer.

12. Migration & Rollback Plan

Migration Path

Step 1: Merge Phase 5.0 (CSV data + StreamingAssets)
 Step 2: Merge Phase 5.1 (DiceExpression – standalone, zero risk)
 Step 3: Merge Phase 5.2 (Parser – standalone, zero risk)
 Step 4: Merge Phase 5.3 (CSV builder + table data changes)
 Step 5: Merge Phase 5.4 (Core integration – this is the critical merge)
 Step 6: Merge Phase 5.5 + 5.6 (UI + testing)

Rollback Strategy

Full rollback: Delete CSV from StreamingAssets → system falls back to hardcoded tables automatically. No code changes needed for immediate rollback.

Partial rollback: Each phase is independently revertable:

- Revert Phase 5.4 → System uses hardcoded tables; new parser files are harmless dead code
- Revert Phase 5.5 → UI stays at levels 1-8; level 9 inaccessible but system still works
- Revert Phase 5.1-5.3 → New files removed; no existing code affected

Feature Flag (Optional)

```
// In DungeonEncounterTableManager:
public static bool UseCSVTables = true; // Set false to force hardcoded tables

public static void LoadTables()
{
    if (UseCSVTables)
    {
        // Try CSV loading...
    }
    // Fallback to hardcoded...
}
```

13. Definition of Done

Phase 5 is complete when ALL of the following are true:

- [] `DiceExpression` class parses all dice patterns found in the CSV
- [] `EncounterDescriptionParser` handles all 7 pattern types (cascade, simple, dice, compound, NPC, annotated, special)
- [] CSV file loads and produces 9 valid encounter tables
- [] `GenerateRandomEncounter()` works for levels 1-9 using CSV data
- [] Dice expressions produce variable creature counts at generation time
- [] Compound entries spawn multiple creature types per encounter
- [] NPC entries resolve to classed creatures
- [] Hardcoded table fallback still works when CSV is absent
- [] UI supports level 1-9 selection
- [] No compile errors or warnings
- [] Stress test: 1000 encounters per level with 0 exceptions
- [] All changes committed to Git with descriptive messages

14. Dependencies & Prerequisites

Must Be Complete Before Phase 5

- [x] Phase 3: Hardcoded encounter tables (levels 1-8) — **DONE**
- [x] Phase 4: Encounter generator UI — **DONE**
- [x] Creature additions: ~90% creature coverage in NPCDatabase — **DONE (88.8%)**
- [x] Name mappings: CSV name → NPCDatabase ID — **DONE (600+ mappings)**
- [x] CSV data file: `dungeon_encounters.csv` — **DONE (267 rows, table 9 partial)**

Nice to Have Before Phase 5

- [] Complete table 9 CSV data (can be done in Phase 5.0)
- [] Remaining ~11% creature coverage (NPCDatabase additions)
- [] EL/CR lookup table for accurate EL calculation

External Dependencies

- Unity 2021+ (for `System.Text.RegularExpressions`)
 - No third-party packages required
-

Appendix: Commit Plan

Commit 1 (Phase 5.0):

- "Phase 5.0: Complete CSV data and prep StreamingAssets"
- `dungeon_encounters.csv` (completed table 9)
- `Assets/StreamingAssets/dungeon_encounters.csv`

Commit 2 (Phase 5.1):

- "Phase 5.1: Add DiceExpression class for dice notation parsing"
- `Assets/Scripts/Encounters/DiceExpression.cs`

Commit 3 (Phase 5.2):

- "Phase 5.2: Add EncounterDescriptionParser for CSV encounter strings"
- `Assets/Scripts/Encounters/EncounterDescriptionParser.cs`

Commit 4 (Phase 5.3):

- "Phase 5.3: Add CSV parser and table builder"
- `Assets/Scripts/Encounters/EncounterCSVParser.cs`
- `Assets/Scripts/Encounters/DungeonEncounterTableData.cs` (modified)

Commit 5 (Phase 5.4):

- "Phase 5.4: Integrate CSV tables into encounter generation pipeline"
- `Assets/Scripts/Encounters/EncounterDefinition.cs` (modified)
- `Assets/Scripts/Encounters/DungeonEncounterTableEntry.cs` (modified)
- `Assets/Scripts/Encounters/DungeonEncounterTableManager.cs` (modified)

Commit 6 (Phase 5.5+5.6):

- "Phase 5.5-5.6: UI update for level 9 and testing"
- `Assets/Scripts/UI/DungeonEncounterGeneratorUI.cs` (modified)
- `Assets/Scripts/Editor/EncounterGeneratorTests.cs` (new, optional)

End of Implementation Plan